

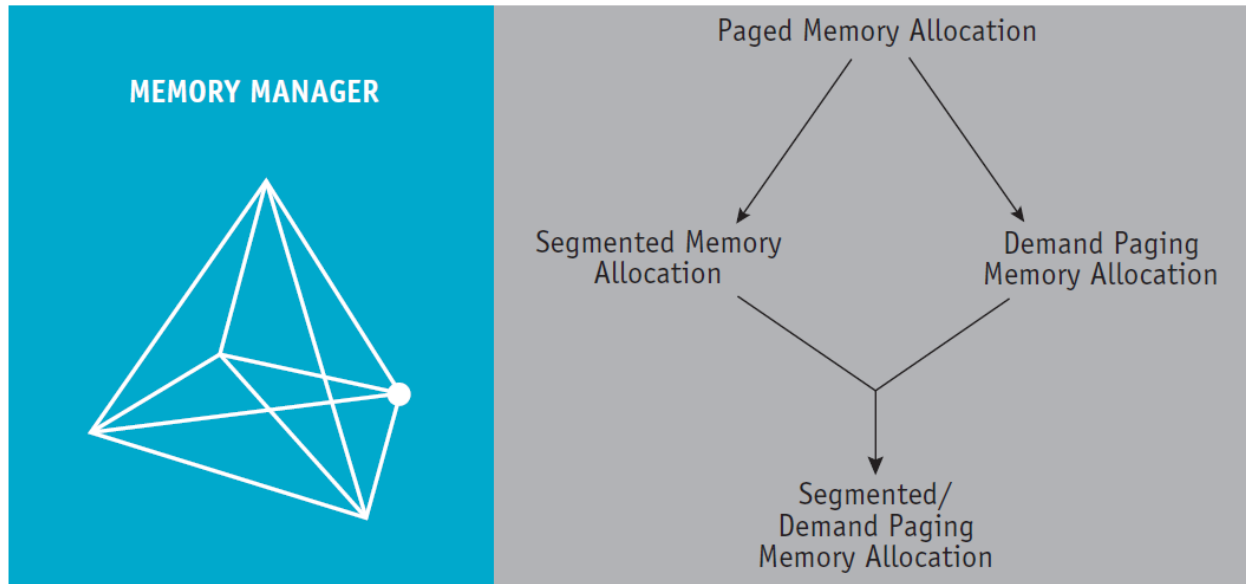
Operating Systems

LECTURE IV: VIRTUAL MEMORY II

Department of Information and Communication Engineering
Institute of Technology of Cambodia



Virtual Memory



“Nothing is so much strengthened by practice, or weakened by neglect, as memory.”

—Quintillian (A.D. 35–100)

Previously...

- ❑ **Page** and **page frame** were introduced.
- ❑ Two replacement policies for demand paging: **FIFO & LRU**.
- ❑ LRU swaps out the pages that show the least recent activity, figuring that these pages are the least likely to be used again in the immediate future.
- ❑ Note for LRU: **X frames – X differences of page.**

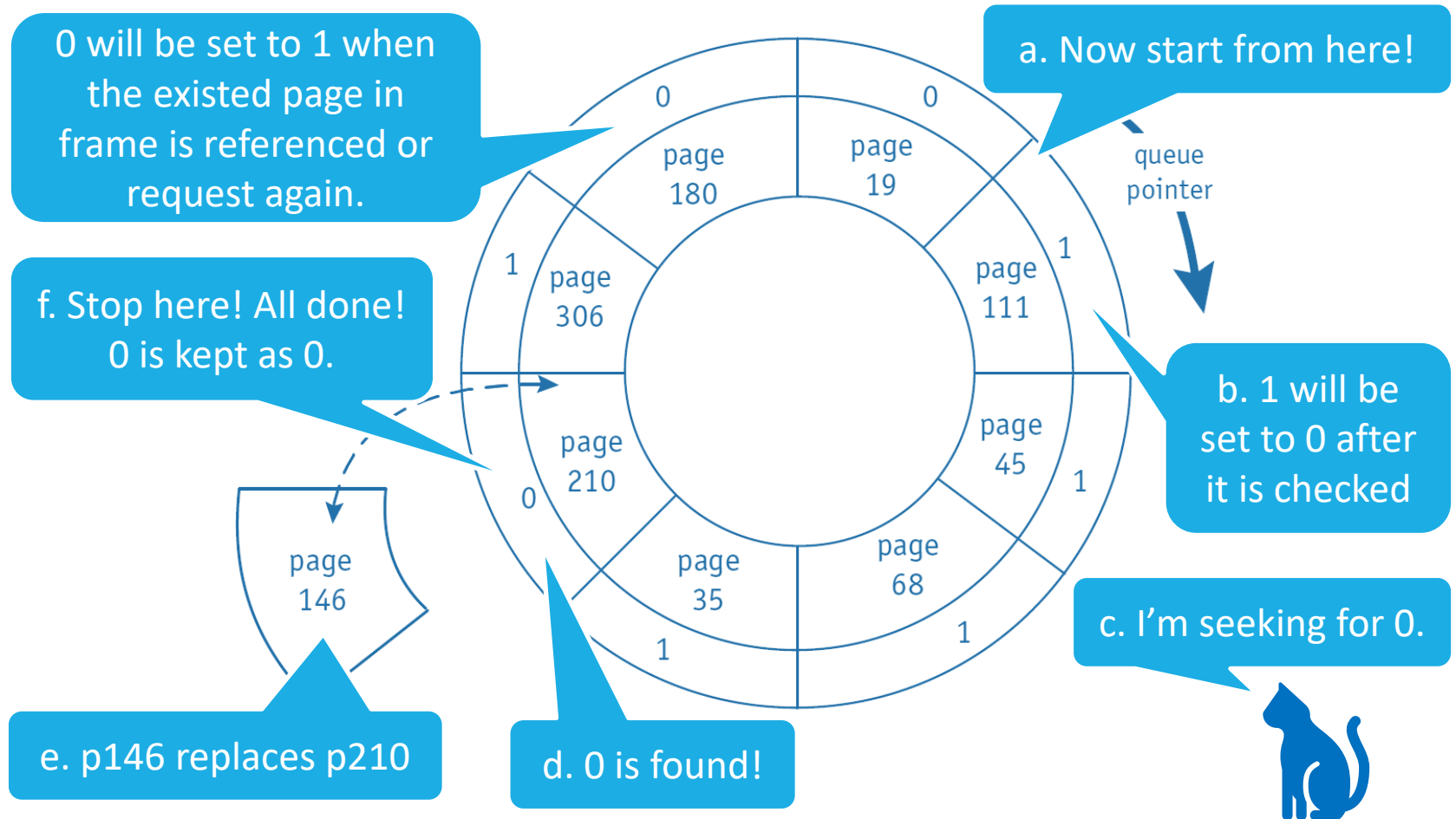


Page	A	B	A	A	C	B
Frame 0	A	A	A	A	A	B
Frame 1		B	B	B	C	C

Clock Replacement Variation

- ❑ A variation of the LRU page replacement is known as the **clock page replacement policy** because it is implemented with a circular queue and uses a pointer to step through the reference bits of the active pages, simulating a clockwise motion.
- ❑ The algorithm checks the reference bit for each page:
 - ❑ If the bit is one (indicating that it was recently referenced), the bit is reset to zero and the bit for the next page is checked.
 - ❑ If the reference bit is zero (indicating that the page has not recently been referenced), that page is targeted for removal.
 - ❑ If all the reference bits are set to one, then the pointer must cycle through the entire circular queue again, giving each page a second and perhaps a third or fourth chance.

Example of Reference Bit of LRU



Bit Shifting Variation

- ❑ A second variation of LRU uses an 8-bit reference byte and a bit-shifting technique to track the usage of each page currently in memory.
- ❑ At specific time intervals of the clock cycle, the Memory Manager shifts every page's reference bytes to the right by one bit, dropping their rightmost bit. Meanwhile, each time a page is referenced, the leftmost bit of its reference byte is set to 1.
- ❑ The page with smallest value in reference bytes will be selected.
- ❑ Example of shifting:
 - ❑ Page A has been referenced for 8 times => 11111111
 - ❑ Page B has not been referenced for 8 times => 00000000

Example of Bit Shifting Variation

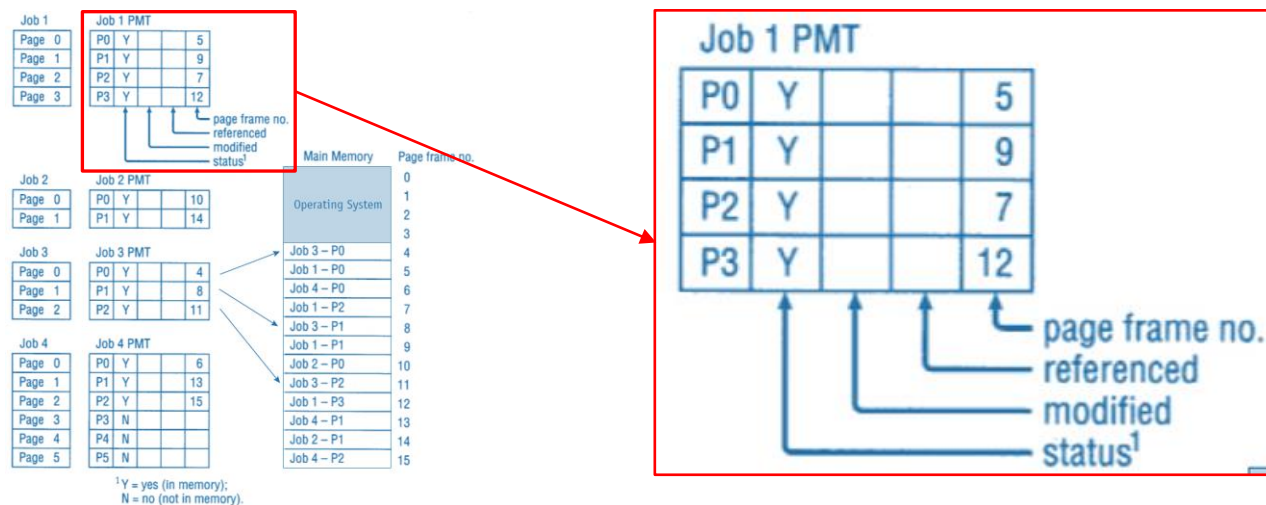
Page Number	Time Snapshot 0	Time Snapshot 1	Time Snapshot 2	Time Snapshot 3	Time Snapshot 4
1	10000000	11000000	11100000	11110000	11111000
2	10000000	01000000	10100000	11010000	01101000
3	10000000	11000000	01100000	00110000	00011000
4	10000000	01000000	10100000	11010000	11101000
5	10000000	11000000	01100000	00110000	00011000
6	10000000	11000000	01100000	10110000	01011000

All pages are referenced.

Page 3 or 5 will be swapped if new page is requested. Memory manager will swap it using other criteria like FIFO or priority or others.

The Mechanics of Paging (1/3)

- Before the Memory Manager can determine which pages will be swapped out, it needs specific information about each page in memory—information included in the Page Map Tables (PMT).



The Mechanics of Paging (2/3)

Page No.	Status Bit	Modified Bit	Referenced Bit	Page Frame No.
0	1	1	1	5
1	1	0	0	9
2	1	0	0	7
3	1	0	1	12

- ❑ **Status bit** indicates whether the page is currently in memory.
- ❑ **Modified bit** indicates whether the contents of the page have been altered; if so, the page must be rewritten to secondary storage when it is swapped out before its frame is released.
- ❑ **Referenced bit** indicates whether the page has been called (referenced) recently.

The Mechanics of Paging (3/3)

	Modified?	Referenced?	What it Means
Case 1	0	0	Not modified AND not referenced
Case 2	0	1	Not modified BUT was referenced
Case 3	1	0	Was modified BUT not referenced [Impossible?]
Case 4	1	1	Was modified AND was referenced

Which should be swapped out first for LRU?

1st : Case 1 because it has not modified and has not referenced.

2nd: Case 3 next but page has been modified, so it takes some time to write to secondary storage.

3rd: Case 2 pages would be the third choice.

4th: Case 4 pages would be those least likely to be removed.



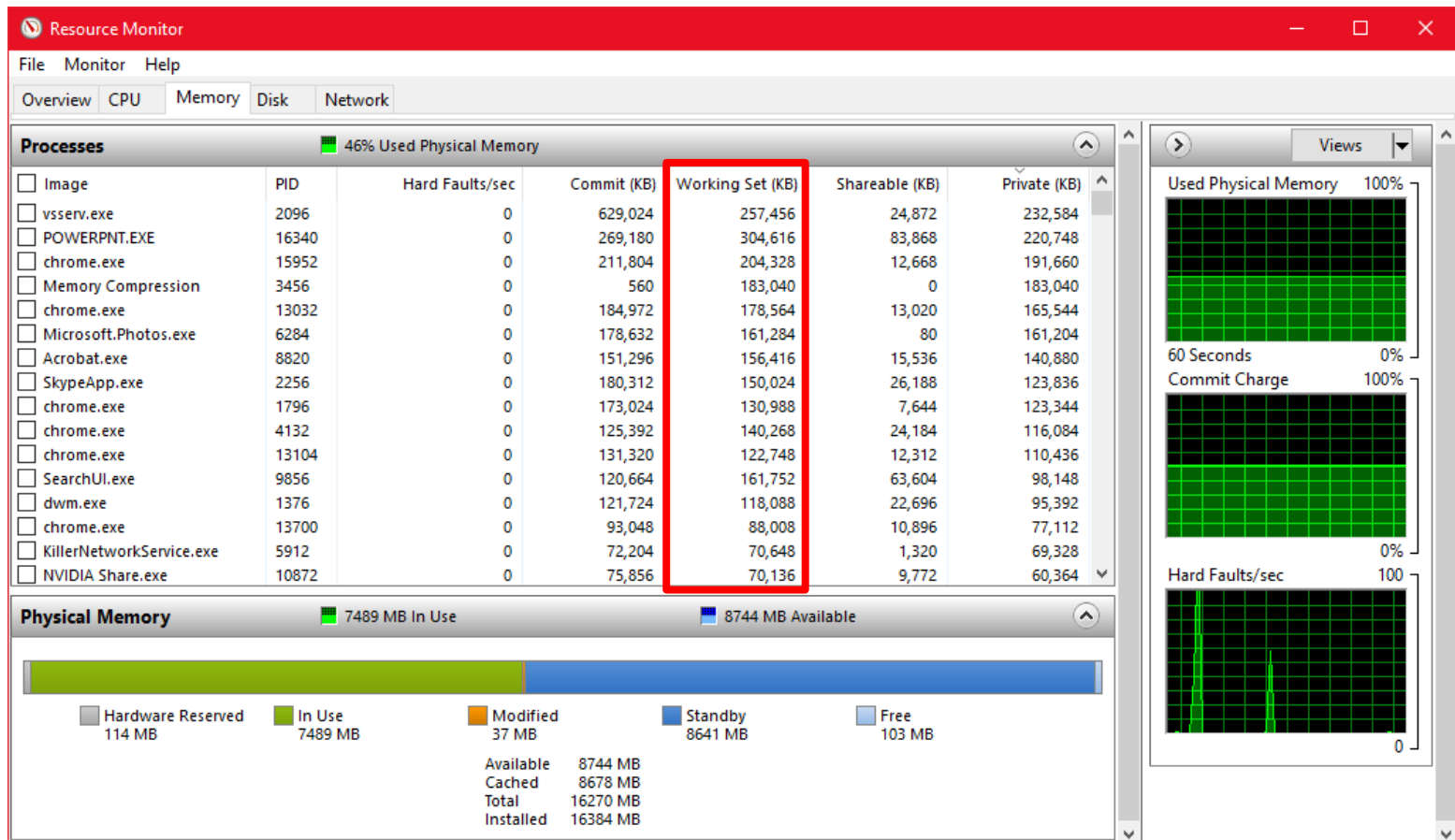
The Working Set

- ❑ An innovation which improves the performance of demand paging schemes, working set, is a collection of pages residing in memory that can be accessed directly without incurring a page fault – published in 1967 – ideal used in many OS from 2005.
- ❑ A job's working set changes as the job moves through the system: one working set could be used to initialize the job, another could work through repeated calculations, another might interact with output devices, and a final set could close the job.
- ❑ Good programs are structured, and this leads to a locality of reference during the program's execution, meaning that during any phase of its execution, the program references only a small fraction of its pages.

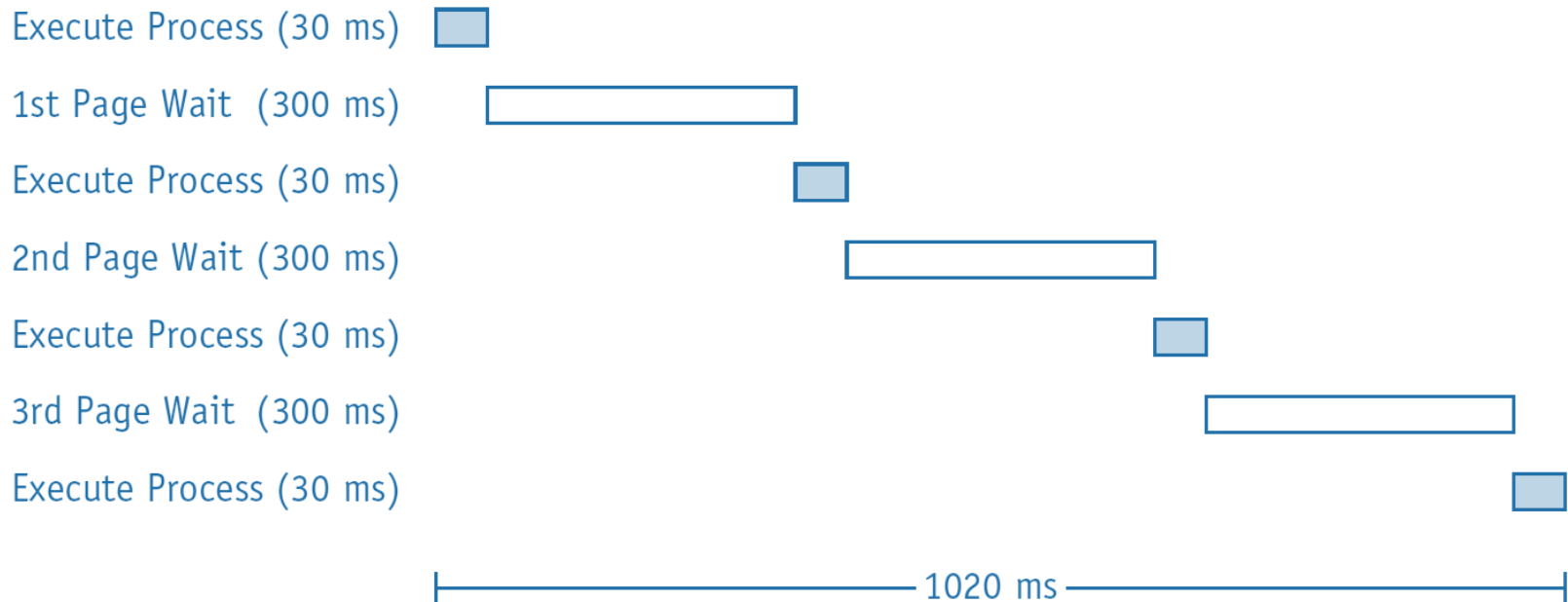
Example of Working Set

- ❑ When a user requests execution of a program:
 - ❑ The first page is loaded into memory and execution continues as more pages are loaded: those containing variable declarations, others containing instructions, others containing data, and so on.
 - ❑ After a while, most programs reach a fairly stable state, and processing continues smoothly with very few additional page faults. At this point, one of the job's working sets is in memory, and the program won't generate many page faults until it gets to another phase requiring a different set of pages to do the work—a different working set.

Working Set on MS. Windows



Page Faults for a Program



Segmentation

Segments take the user's view of the program and gives it to the OS.

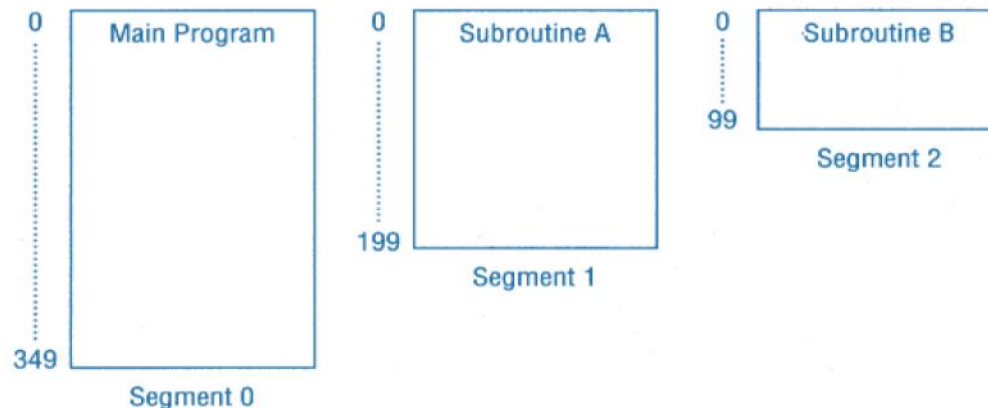
- User views the program in logical segments, e.g., code, global variables, stack, heap (dynamic data structures), not a single linear array of bytes.

Segmented Memory Allocation

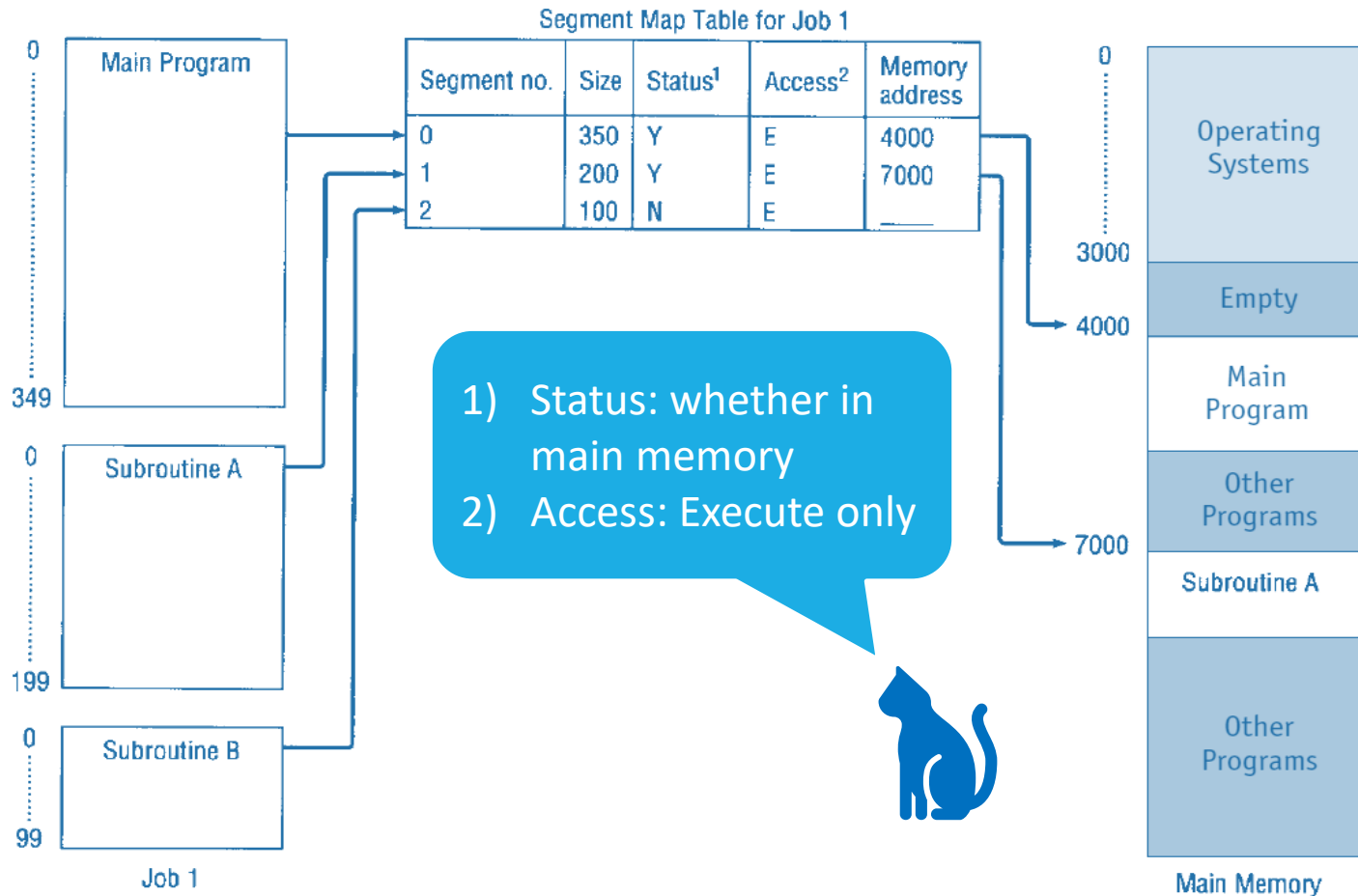
- ❑ The concept of segmentation is based on the common practice by programmers of structuring their programs in modules—logical groupings of code.
- ❑ With segmented memory allocation, each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions.
- ❑ A subroutine is an example of one such logical group. Segmented memory allocation was designed to reduce page faults.
- ❑ Main memory is no longer divided into page frames, because the size of each segment is different ranging from quite small to large.

Segmentation

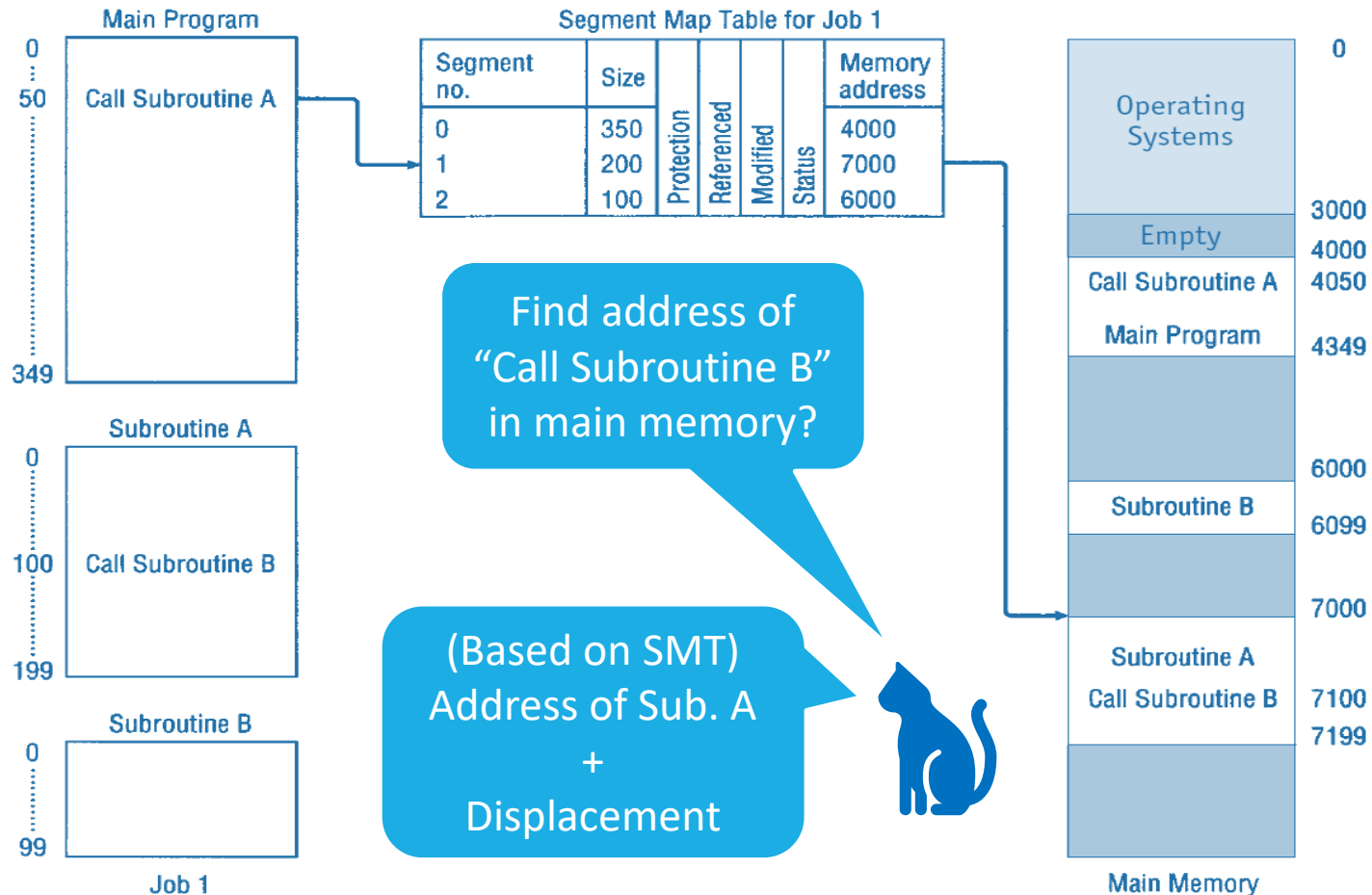
- ❑ When a program is compiled or assembled, the segments are set up according to the program's structural modules.
- ❑ Each segment is numbered and a Segment Map Table (SMT) is generated for each job; it contains the segment numbers, their lengths, access rights, status, and (when each is loaded into memory) its location in memory



Seg. Mem. All. – Example 1



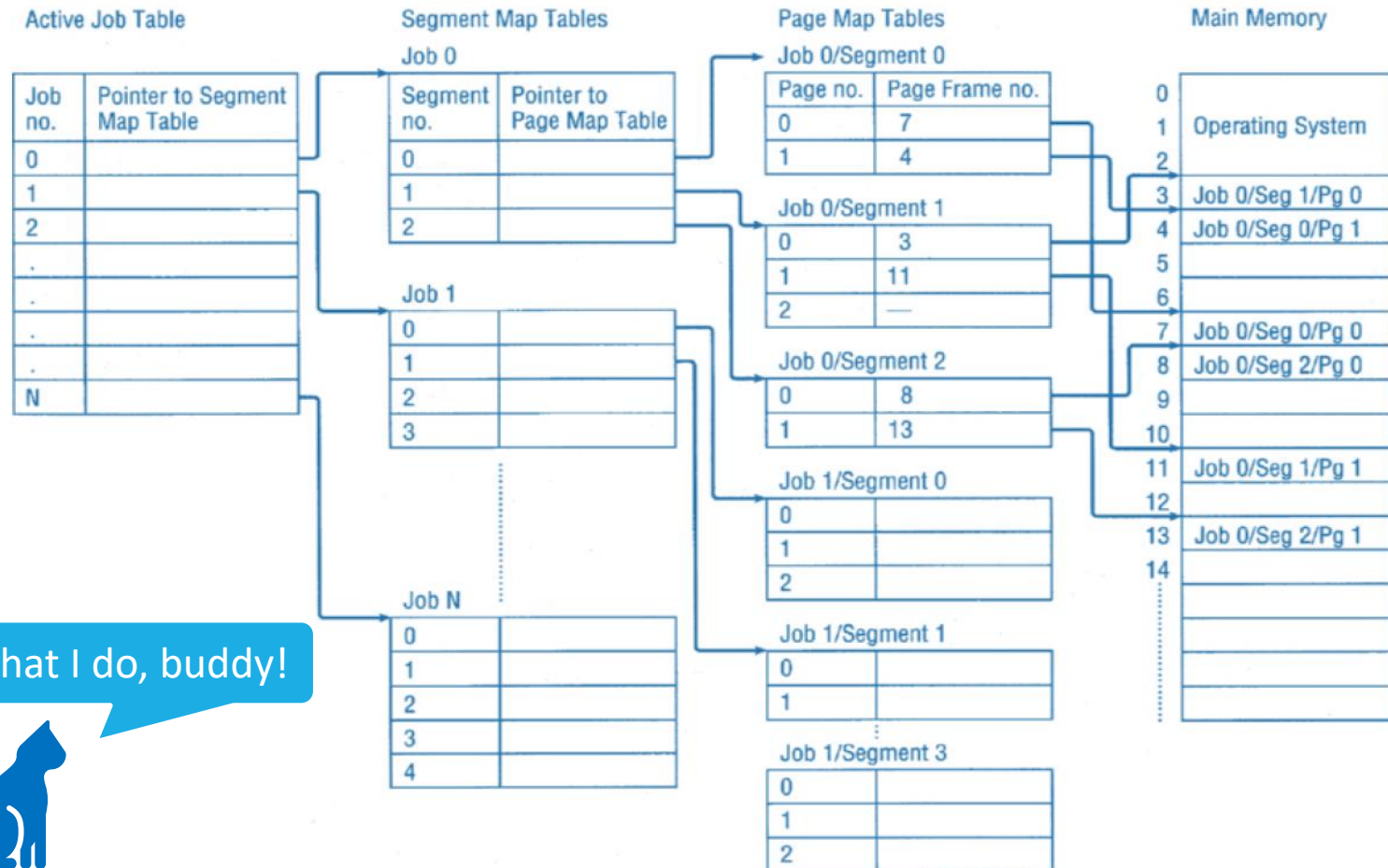
Seg. Mem. All. – Example 2



Segmented/Demand Paged Memory Allocation

- ❑ The segmented/demand paged memory allocation scheme evolved from the last two schemes.
- ❑ It is a combination of segmentation and demand paging, and it offers the logical benefits of segmentation, as well as the physical benefits of paging.
- ❑ This allocation scheme doesn't keep each segment as a single contiguous unit, but subdivides it into pages of equal size that are smaller than most segments and more easily manipulated than whole segments. Therefore, many of the problems of segmentation (compaction, external fragmentation, and secondary storage handling) are removed because the pages are of fixed length.

Segmented/Demand Paged Memory Allocation – Example



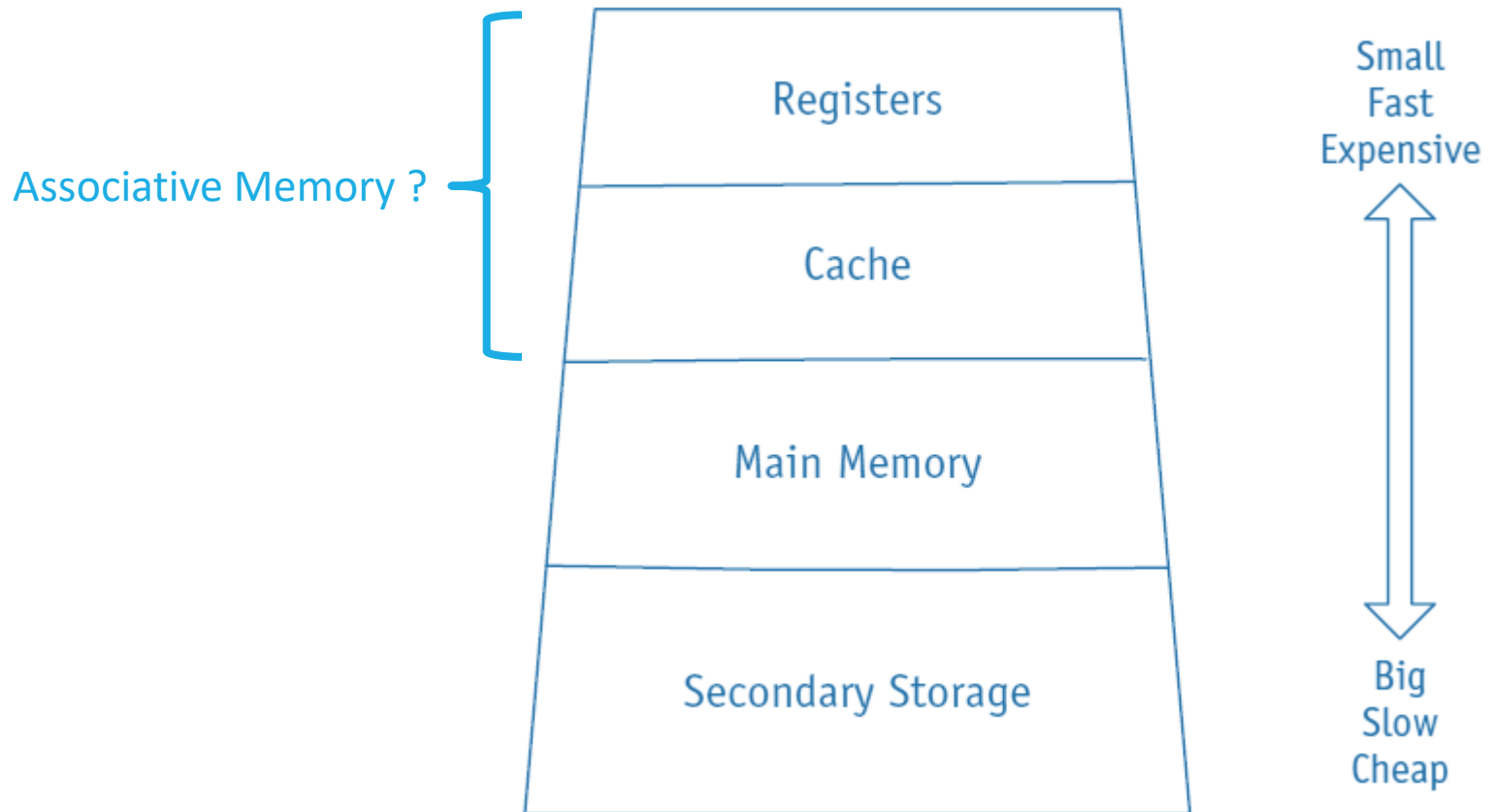
That's what I do, buddy!



Segmented/Demand Paged Memory Allocation – Drawback

- ❑ The overhead that is required to manage the tables (Segment Map Tables and the Page Map Tables)
- ❑ The time required to reference them. To minimize the number of references, many systems take advantage of associative memory to speed up the process.
- ❑ **Associative memory?** It is a name given to several registers that are allocated to each job that is active. Their task is to associate several segment and page numbers belonging to the job being processed with their main memory addresses. These associative registers are hardware, and the exact number of registers varies from system to system.

Memories & Associative Memory



Homework

1. Exercise 7 on page 97 of ref. book.